

2026-06-24 — Search Timeout Coordination Analysis

Read-only investigation supporting the concurrent fix streams:

- **Engine fix:** add a per-handler deadline so `GET /v1/{provider}/search` always returns before the client read-timeout fires.
- **Client fix:** make the in-flight search job cancellable on back-press.

This document establishes the **exact numbers**, **scope verdict**, and **recommended coordinated timeout values** with file:line evidence for every claim.

1. Scope verdict: yts-specific bug or general slow-provider bug?

Verdict: GENERAL — any provider whose upstream is reachable but slow from the phone's mobile network can trigger the same `SocketTimeoutException`.

Evidence:

Provider	DefaultTimeout	mirrors	perAttemptTimeout	Can exceed 30s?
YTS	20s (:yts/client.go:46)	4 (:yts/client.go:35-40)	8s (:yts/client.go:51)	YES — $4 \times 8s = 32s$ theoretical
ThePirateBay	20s (:thepiratebay/client.go:59)	1 (:thepiratebay/client.go:51-53)	8s (:thepiratebay/client.go:64)	No ($1 \times 8s \leq 20s$ 30s)
TorrentsCSV	20s (:torrentscsv/client.go:66)	1 (:torrentscsv/client.go:55-57)	8s (:torrentscsv/client.go:71)	No ($1 \times 8s \leq 20s$ 30s)
BitSearch	20s (:bitsearch/client.go:36)	1	— (no mirror failover)	No (20s 30s)
Knaben	20s (:knaben/client.go:43)	1	—	No
Nyaa	20s (:nyaa/client.go:39)	1	—	No
TokyoToshokan	20s (:tokyotosho/client.go:49)	1	—	No

Provider	DefaultTimeout	mirrors	perAttemptTimeout	Can exceed 30s?
TorrentDownloads	20s (:torrentdownloads/client.go:37)	1	—	No
rutracker	30s (:rutracker/client.go:163)	N/A (single)	—	YES — 30s == client timeout (zero slack)
kinozal	30s (:kinozal/client.go:117)	N/A	—	YES — 30s == client timeout
nnmclub	30s (:nnmclub/client.go:113)	N/A	—	YES — 30s == client timeout
archiveorg	30s (:archiveorg/client.go:34)	N/A	—	YES — zero slack
gutenberg	30s (:gutenberg/client.go:42)	N/A	—	YES — zero slack

Why YTS surfaces this first: YTS has 4 mirrors iterated sequentially, and the primary mirror yts.mx was observed NXDOMAIN on public DNS on 2026-06-13 (lava-api-go/internal/provider/curated/yts/client.go:28–30). On a mobile network, DNS resolution + TCP handshake + slow/blocked yts.mx can consume the full perAttemptTimeout=8s before failing over to the next mirror, accumulating $\approx 8s \times N$ -slow-mirrors of latency before a working mirror responds.

The 5 non-curated providers (rutracker, kinozal, nnmclub, archiveorg, gutenberg) all use `http.Client.Timeout=30s` with zero handler-level deadline — a single slow upstream response hits the Android client 30s read timeout exactly, with no margin.

Conclusion: The bug class is "provider upstream slow/stale from a mobile network hits the Android OkHttp read timeout." YTS triggered it first due to its 4-mirror failover + a known stale primary domain; the same failure mode applies to any single-provider search (GET /v1/{provider}/search) against a slow provider. The GET /v1/multi path (SSE streaming) is a separate concern documented in §4 below.

2. Full timeout chain, end to end

Android client layer

File: core/network/impl/src/main/kotlin/lava/network/di/NetworkModule.kt

```
.connectTimeout(30, TimeUnit.SECONDS)    // line 173
.readTimeout(30, TimeUnit.SECONDS)       // line 174
.writeTimeout(30, TimeUnit.SECONDS)      // line 175
```

This is the `@Named("lan") OkHttpClient` used for all calls to the mDNS-discovered local engine. No `callTimeout()` is configured (a `callTimeout` would cap the entire roundtrip including connection; its absence means the only bound is the per-phase `readTimeout/writeTimeout/connectTimeout`). The client is shared by all API calls including `GET /v1/{provider}/search` and `GET /v1/multi`.

HTTP/2 server layer

File: `lava-api-go/internal/server/server.go`

```
ReadHeaderTimeout: 5 * time.Second,    // line 83 (h2srv) and line
                                     88 (metrics srv)
```

`ReadHeaderTimeout=5s` applies only to reading the incoming request headers from the Android client. There is **no** `ReadTimeout`, `WriteTimeout`, or `IdleTimeout` on the server — response time is unbounded at the transport layer. The 5s header-read guard is appropriate; the missing response deadline is the gap.

Single-provider search handler: `GET /v1/{provider}/search`

File: `lava-api-go/internal/handlers/v1/search.go`

```
result, err := p.Search(c.Request.Context(), opts, creds)    //
                                     line 64
```

`c.Request.Context()` is the raw Gin request context — it carries the HTTP transport's context (which cancels on client disconnect) but **no additional deadline**. If the client stays connected (which it does until `OkHttp` fires its own `readTimeout`), `p.Search()` runs until the provider's own `http.Client.Timeout` expires. There is no handler-level `context.WithTimeout` wrapping the call.

Multi-search handler: `GET /v1/multi`

File: `lava-api-go/internal/handlers/v1/search.go`

```
ctx, cancel := context.WithTimeout(c.Request.Context(),
                                     30*time.Second)    // line 201
```

The multi-search handler does apply a per-provider deadline of 30s. However this is sequential (providers are processed one at a time inside `c.Stream()` at line 156). See §4 for the streaming/architecture analysis.

YTS provider layer

File: `lava-api-go/internal/provider/curated/yts/client.go`

```

const DefaultTimeout      = 20 * time.Second // line 46 (entire
    http.Client cap)
const perAttemptTimeout = 8 * time.Second // line 51 (per-
    mirror failover cap)

```

With 4 mirrors (lines 35-40) and the primary yts.mx DNS-dead or slow, the worst-case path:

```

attempt 1 (yts.mx): 8s timeout fires → fail, continue
attempt 2 (yts.bz): 8s timeout fires → fail, continue
attempt 3 (yts.lt): http.Client.Timeout=20s remaining cap; if <
20s elapsed,
                        this attempt gets a bounded context; if > 20s
elapsed,
                        ctx is already cancelled
attempt 4 (yts.am): similar

```

The `http.Client.Timeout=20s` wraps the ENTIRE `Search()` call (the whole mirror loop, not per attempt). The `perAttemptTimeout=8s` bounds each mirror individually **only when `len(baseURLs) > 1`** (line 150). So maximum total latency from the yts `Search()` call is bounded at `DefaultTimeout=20s` by the `http.Client.Timeout`.

However, `http.Client.Timeout` is enforced by the Go HTTP runtime, which cancels the context after 20s. The handler at line 64 has **no additional deadline** — so the 20s yts timeout is the only bound. Since `20s < 30s`, yts alone cannot exceed the client read timeout in theory.

What actually causes the >30s `SocketTimeoutException`:

The `http.Client.Timeout` on the `http.Client` value at `lava-api-go/internal/provider/curated/yts/client.go:92`:

```

http: &http.Client{Timeout: DefaultTimeout}, // line 92

```

...only applies when the `Search()` call uses that `http.Client` directly. The `perAttemptTimeout` is a `context.WithTimeout` wrapping each mirror attempt (line 151). BUT the outer handler context at `search.go:64` (`c.Request.Context()`) is passed **into** `p.Search()` as the parent context. If `c.Request.Context()` itself has a long-lived deadline (it doesn't — it's the raw Gin/HTTP2 context), the `http.Client.Timeout` is the true bound.

The real failure scenario confirmed by the Crashlytics `SocketTimeoutException` at `Http2Stream.takeHeaders` (not at read or write): the problem is that `OkHttp` sent the HTTP/2 headers for `GET /v1/yts/search` and then waited 30s for the response headers — the Go server accepted the connection but the `Search()` call took close to or over 20s (e.g. attempting dead mirrors, or slow from this specific HUAWEI device's mobile network), causing `OkHttp`'s `readTimeout` to fire first.

The 30s `OkHttp readTimeout` == 20s provider timeout + ~10s network overhead headroom is razor-thin on a slow mobile network. Any combination of:

- slow DNS for one mirror
- TCP SYN timeout (often 30s on mobile carriers)
- slow TLS handshake

- HTTP keep-alive reuse with a stale server connection

...can push the actual elapsed response time past 30s before Go delivers the response headers.

Summary of where the >30s comes from:

Android OkHttp readTimeout:	30s (NetworkModule.kt:174)
Go handler deadline:	NONE (search.go:64)
YTS http.Client.Timeout:	20s (yts/client.go:46)
YTS perAttemptTimeout:	8s (yts/client.go:51)
YTS mirrors:	4 (yts/client.go:35-40)
Go server ReadHeaderTimeout:	5s (server.go:83, inbound only)
Network overhead (mobile):	variable

Worst case: 20s (yts search) + variable network jitter > 30s
OkHttp readTimeout
→ SocketTimeoutException at Http2Stream.takeHeaders

3. Recommended coordinated timeout values

The root problem is zero margin between the engine's max response time and the client's read timeout. The fix must establish a strict ordering:

provider_http_timeout < handler_deadline < client_readTimeout

Recommended values

Layer	Current	Recommended	File	Rationale
YTS DefaultTimeout	20s	15s	yts/client.go:46	Reduces max yts search time to 15s (from 32s) but outer cap at 15s
YTS perAttemptTimeout	8s	5s	yts/client.go:51	4 mirrors × 5s = 20s max search time; outer cap; dead mirror timeout
TPB/TorrentsCSV perAttemptTimeout	8s	5s	respective client.go	Align with YTS; single mirror; unaffected in practice
GetSearch handler deadline	none	20s	search.go:64	context.WithTimeout(20*time.Second) before engine ret; guarantees engine returns error, leaving 10s slack
GetMultiSearch per-provider deadline	30s	20s	search.go:201	Match the single-provider deadline; max per-provider wait
Android OkHttp readTimeout (LAN)	30s	45s	NetworkModule.kt:174	Raise above the engine's max response time; 10s+ network margin; engine can respond
Android OkHttp callTimeout (LAN)	none	120s	NetworkModule.kt (new)	For multi-search SSE; callTimeout to cap the total time; prevents infinite hang

Layer	Current	Recommended	File	Rationale
Go server WriteTimeout	none	25s	server.go	Cap the server's response time prevents zombie connections

The key invariant:

engine handler deadline (20s) + network round-trip margin (5–10s)
 < client readTimeout (45s)

With these values:

- A dead provider times out in the engine at 20s, the engine returns a `provider_error`, and the client receives the response at $\sim 20\text{s} + \text{network latency}$ — well under 45s.
- The client `readTimeout=45s` fires only if the engine itself hangs (which is now bounded by the handler deadline and the server `WriteTimeout`).
- The `callTimeout=120s` on the Android LAN client caps multi-search SSE at 120s total regardless of how many providers are selected.

Whether to add Android `callTimeout`

A `callTimeout` in `OkHttp` limits the entire call (connect + send + receive). For SSE (multi-search), this is appropriate because the stream is open for $N \times 20\text{s}$. For single-provider search, `callTimeout=120s` is a safety backstop only — the handler deadline of 20s dominates. Adding `callTimeout` is LOW RISK and HIGH VALUE for the long-running SSE case.

4. On-device reachability: is YTS (or other providers) reachable from a typical mobile network?

Cloudflare / FlareSolvrr wiring

All 8 curated providers are documented as **"anonymous, no Cloudflare"** in their package comments:

- `bitsearch/client.go:2` — "anonymous, no Cloudflare"
- `knaben/client.go:3` — "no Cloudflare challenge"
- `nyaa/client.go:2` — "anonymous, no Cloudflare"
- `thepiratebay/client.go:2` — "anonymous, no Cloudflare"
- `tokyotosho/client.go:3` — "no Cloudflare, no login"
- `torrentdownloads/client.go:3` — "no Cloudflare challenge"
- `torrentscsv/client.go:3` — "no Cloudflare"
- `yts/client.go:2` — "anonymous, no Cloudflare"

The `lava-api-go/internal/provider/flaresolvrr/` package exists and is implemented (with `DefaultMaxTimeout=60000ms` and `DefaultTimeout=90s`) but is **not wired to any currently active provider**. It is reserved for future CF-gated providers (e.g. 1337x). YTS does NOT need FlareSolvrr.

Mobile network reachability

The YTS primary mirror `yts.mx` was observed NXDOMAIN on public DNS on 2026-06-13 (`yts/client.go:28-30`; forensic evidence at `.lava-ci-evidence/bluff-hunt/2026-06-13-yts-domain-failover.json`). Mobile ISPs in Russia (where the operator's testers are) may additionally apply DNS-level blocking for torrent sites. The 4-mirror failover (`yts.mx`, `yts.bz`, `yts.lt`, `yts.am`) mitigates this — `yts.lt` and `yts.am` were confirmed serving HTTP 200 on 2026-06-13.

Implication for UX: if the phone's DNS blocks all 4 mirrors, the correct UX is a FAST "provider unavailable" message (not a 30s hang). The recommended 20s handler deadline + 5s `perAttemptTimeout` means a fully-blocked YTS produces an error in ~20s (4 mirrors × 5s) rather than silently hanging until `OkHttp` fires.

Further improvement (out of scope for the immediate fix): a fast pre-flight DNS check or a configurable "skip unreachable providers" flag in the API-app preferences — returning "provider unavailable" in <1s for DNS-blocked domains.

knaben note

`knaben/client.go:3` notes "no Cloudflare challenge" but also "indirectly reaches CF-gated aggregated trackers" — Knaben aggregates other trackers, some of which use Cloudflare. This does not affect Knaben's own API reachability but may affect result quality if the aggregated trackers are blocked.

5. Streaming assessment: GET /v1/{provider}/search vs GET /v1/multi

Single-provider search (GetSearch)

File: `lava-api-go/internal/handlers/v1/search.go:33-76`

Response mode: **ALL-AT-ONCE JSON** — the handler calls `p.Search()` at line 64, then `json.Marshal(result)` at line 69, then `c.Data(http.StatusOK, ...)` at line 75. No chunked transfer encoding, no streaming. The client waits for the complete 200 OK with the full JSON body before `OkHttp` delivers it to the app.

This means: if `p.Search()` takes 20s, the Android client sees 20s of silence (no response headers, no body) before the complete response arrives. `OkHttp`'s `readTimeout` fires if this silence exceeds 30s.

Multi-search (GetMultiSearch)

File: `lava-api-go/internal/handlers/v1/search.go:115-~260`

Response mode: **SSE (Server-Sent Events)** — `text/event-stream` header at line 146; events flushed via `streamEvent()` which calls `f.Flush()` at `search.go:93` after each event. Events emitted: `provider_start`, `results`, `provider_done`, `provider_error`, `stream_end`.

In SSE mode, each flushed event resets OkHttp's read-idle timer. However **processing is sequential** (line 156: `for _, pid := range providerIDs`). A slow first provider delays the `provider_start` event for the second provider; the Android client sees a stream of events but with gaps up to 30s between them.

Would streaming improve UX for single-provider search?

Yes, but it is architectural work. Options in order of complexity:

1. **Immediate (fix in scope):** Add a `context.WithTimeout` deadline in the handler (20s) so `p.Search()` is bounded. Return fast error, not hang.
2. **Near-term:** Convert `GetSearch` to stream results as they arrive from the provider (not applicable for yts which returns all results in one JSON response — only relevant for providers that paginate server-side).
3. **Longer-term:** Parallelize multi-search providers in `GetMultiSearch` — run all providers concurrently (fan-out) instead of sequentially. With N providers and 20s per-provider deadline, sequential = $N \times 20s$; parallel = $\max(20s)$. For 13 providers: 260s sequential vs 20s parallel.

The **immediate fix** (adding the 20s handler deadline) is the right scope for the current fix streams. It eliminates the `SocketTimeoutException` while keeping the response shape identical.

6. Three strongest file:line citations

1. `core/network/impl/src/main/kotlin/lava/network/di/NetworkModule.kt:174` — `readTimeout(30, TimeUnit.SECONDS)` — the Android LAN client's 30s read timeout that fires before the engine responds.
 2. `lava-api-go/internal/handlers/v1/search.go:64` — `result, err := p.Search(c.Request.Context(), opts, creds)` — the single-provider handler calls `Search` with no deadline; this is the missing `context.WithTimeout` that the engine-side fix must add.
 3. `lava-api-go/internal/provider/curated/yts/client.go:46,51` — `DefaultTimeout = 20s / perAttemptTimeout = 8s` — YTS's mirror-failover arithmetic ($4 \text{ mirrors} \times 8s = 32s$ possible but bounded at 20s by `http.Client`) clarifies why mobile network jitter pushes total latency over 30s.
-

Summary

Item	Value
Scope verdict	GENERAL — any provider whose upstream is slow from the phone's mobile network; YTS surfaces it first due to 4-mirror failover + DNS-dead primary mirror 20s (<code>context.WithTimeout</code> added at <code>search.go:64</code>)

Item	Value
Recommended engine handler deadline (GetSearch)	
Recommended engine per-provider deadline (GetMultiSearch)	20s (reduce from 30s at <code>search.go:201</code>)
Recommended Android LAN readTimeout	45s (raise from 30s at <code>NetworkModule.kt:174</code>)
Recommended Android LAN callTimeout	120s (new; add to <code>NetworkModule.kt</code> builder)
YTS DefaultTimeout	15s (lower from 20s)
YTS/TPB/TorrentsCSV per AttemptTimeout	5s (lower from 8s)
FlareSolverr needed for YTS?	No — YTS is "anonymous, no Cloudflare"
Single-provider search response mode	All-at-once JSON (no streaming)
Multi-search response mode	SSE streaming, sequential per-provider